HLD - High Level Design

1. System Overview

Problem Statement:

Traditional customer support chatbots often rely solely on large language models (LLMs). While LLMs are powerful, they can hallucinate or generate incorrect answers when they lack grounding in factual data. This creates a trust gap between customers and automated systems. In industries like airlines, healthcare, and enterprise software, inaccurate responses can lead to confusion, dissatisfaction, or even financial loss.

The challenge is to design a system that ensures factual, contextual, and reliable answers by combining LLMs with a structured knowledge base. Retrieval-Augmented Generation (RAG) addresses this by retrieving relevant information from a document store before generating answers.

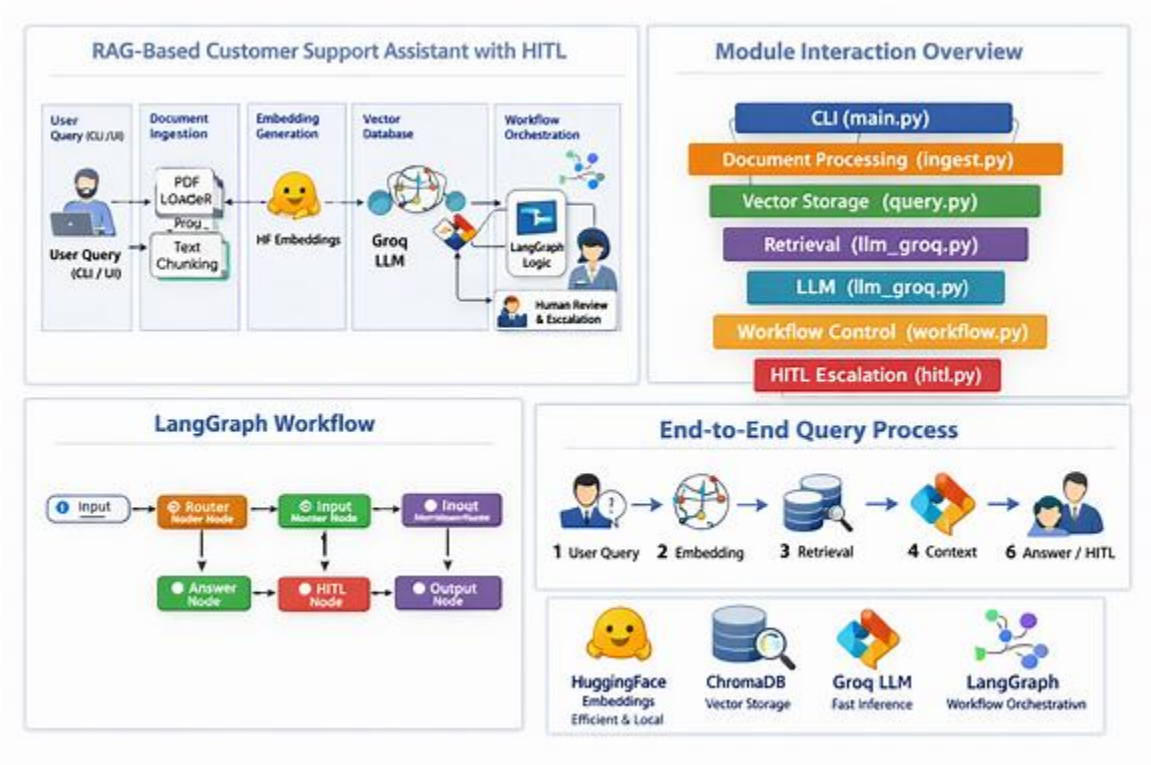
Scope:

This project builds a Customer Support Assistant that:

- Ingests a PDF knowledge base.
- Splits the document into manageable chunks.
- Converts chunks into embeddings stored in ChromaDB.
- Retrieves relevant chunks for user queries.
- Uses Groq LLM to generate contextual answers.
- Employs LangGraph for workflow orchestration.
- Escalates unresolved queries to human agents via HITL.

The assistant supports both CLI and Streamlit UI interfaces, making it suitable for academic demonstration and real-world deployment.

2. Architecture Diagram



Explanation:

- User Query (CLI/UI): Entry point for customer questions.
- Document Ingestion: PDF Loader extracts text; Chunker splits into segments.
- Embedding Generation: HuggingFace embeddings convert text into vectors.
- Vector Database (ChromaDB): Stores embeddings for semantic search.
- Retriever: Finds top-k relevant chunks.
- LLM (Groq): Generates contextual answers.
- LangGraph Workflow: Controls routing between nodes.
- HITL Module: Escalates low-confidence queries to human agents.
- Response: Delivered back to the user.

3. Component Descriptions

Document Loader

Uses PyPDFLoader to extract text from the PDF knowledge base. Ensures structured ingestion of large documents.

Chunking Strategy

Applies RecursiveCharacterTextSplitter with chunk size = 1000 characters, overlap = 200. This balances retrieval accuracy and context preservation.

Embedding Model

Uses HuggingFace MiniLM-L6-v2 embeddings. These are lightweight, free, and local, avoiding dependency on OpenAI keys.

Vector Store

ChromaDB stores embeddings persistently. It supports semantic similarity search, enabling efficient retrieval of relevant chunks.

Retriever

Performs similarity search to fetch top-k chunks. Ensures that answers are grounded in factual context.

LLM

Groq LLaMA 3 (updated model) generates answers using retrieved context. Groq provides fast inference, making it suitable for real-time support.

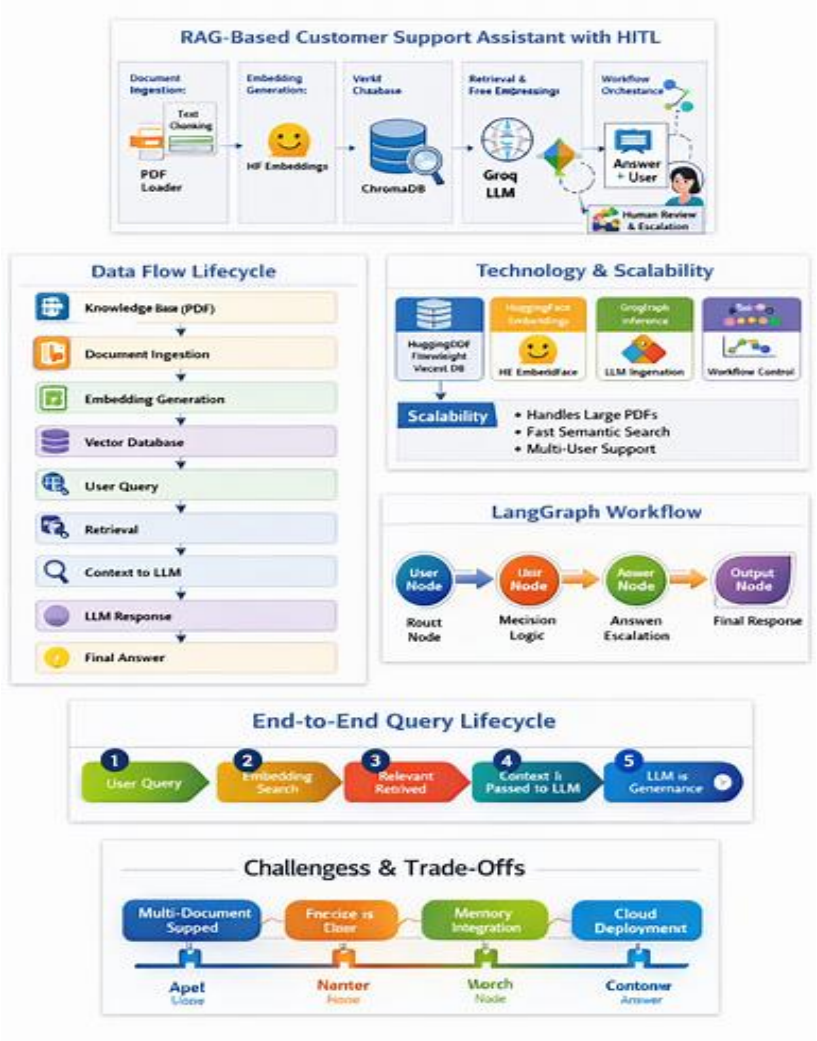
Workflow Orchestration (LangGraph)

Defines nodes and edges for query processing. Controls conditional routing between AI answers and HITL escalation.

HITL Module

Handles escalation when confidence is low or context is missing. Human agents provide manual answers, ensuring reliability.

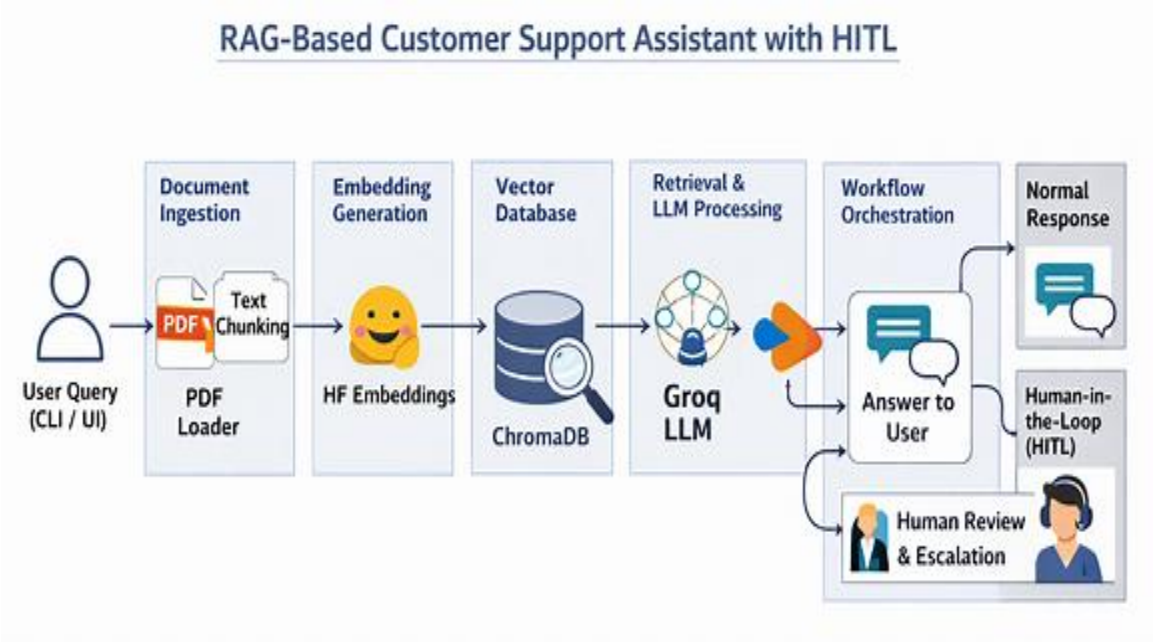
4. Data Flow



Lifecycle:

1. PDF → Loader → Text.
2. Text → Chunker → Segments.
3. Segments → Embeddings → Vectors.
4. Vectors → Stored in ChromaDB.
5. User Query → Converted to embedding.
6. Retriever → Fetches top-k chunks.
7. Groq LLM → Generates contextual answer.
8. LangGraph → Routes answer or escalates to HITL.
9. User receives final response.

5. Technology Choices



- **ChromaDB:** Lightweight, persistent vector DB.
- **HuggingFace Embeddings:** Free, local, avoids API dependency.
- **Groq LLM:** Fast inference, scalable.
- **LangGraph:** Workflow orchestration with conditional routing.

6. Scalability Considerations

- **Handling Large Documents**
  Chunking strategy ensures large PDFs can be processed efficiently.
- **Increasing Query Load**
  ChromaDB supports scaling with larger datasets and concurrent queries.
- **Latency Concerns**
  Groq LLM provides low-latency inference. HuggingFace embeddings are lightweight, reducing processing time.
- **Multi-User Support**
  System can be extended to handle multiple users simultaneously via Streamlit UI or API endpoints.

Conclusion

The High-Level Design defines a modular, scalable architecture for a RAG-based Customer Support Assistant. By combining document ingestion, embeddings, vector storage, retrieval, LLM processing, workflow orchestration, and HITL escalation, the system ensures factual, contextual, and reliable customer support.